

AI Performance Engineering

From FLOPs to Fast Transformers

Cost models • Memory hierarchy • Roofline • Profiling • Scaling laws

Dong Li
UC Merced

Learning goals

Predict cost, locate bottlenecks, and recommend optimizations.

Estimate

parameters, bytes, FLOPs, and training time

Classify

compute-bound, bandwidth-bound, or overhead-bound

Measure

kernels with a scientific benchmark protocol

Optimize

batching, tiling, fusion, precision, and data reuse

THINK Day 2 starts only after local cost and bottlenecks are understood.

01

Parameters and memory

Count tensors before estimating performance.

Worked example: dense-layer parameters

$$Y = XW + b$$

Dimensions

input = 1,024
output = 4,096

Weights

$1,024 \times 4,096$
 $= 4,194,304$

Bias

4,096 parameters

Total = 4,198,400 parameters

BF16 storage

≈ 8.4 MB in decimal units

FP32 storage

≈ 16.8 MB

Model storage by precision

Parameter files are only one part of training memory.

Model	FP32 GB	BF16 GB	INT8 GB
1B	4	2	1
7B	28	14	7
13B	52	26	13
70B	280	140	70

Inference question

Can the parameters fit?

Training question

What about gradients, states, activations, buffers?

THINK A 70B BF16 model requires about 140 GB for parameters alone.

Approximate transformer parameter count

For a standard dense block with FFN width near 4d.

Attention projections

Q, K, V, output
 $\approx 4d^2$

Feed-forward network

up + down projections
 $\approx 8d^2$

One block

$\approx 12d^2$ parameters

$$\text{Nblocks} \approx 12 L d^2$$

Worked result

$L = 24, d = 1,024 \rightarrow$ approximately 302 million block parameters

THINK Doubling depth doubles block parameters; doubling width quadruples them.

02

Training compute

Turn model and data scale into FLOPs and time.

A high-level transformer training estimate

Useful for planning and comparison, not an exact profiler.

$$C \approx 6ND$$

N

number of model parameters

D

number of training tokens

6

forward + backward planning constant

THINK The constant depends on architecture and implementation.

Worked example: training time

1B parameters trained on 20B tokens.

Compute

$$6 \times 10^9 \times 20 \times 10^9 \\ = 1.2 \times 10^{20} \text{ FLOPs}$$

Ideal hardware

$$100 \text{ TFLOP/s sustained} \\ = 10^{14} \text{ FLOP/s}$$

Ideal time

$$1.2 \times 10^6 \text{ seconds} \\ \approx 13.9 \text{ days}$$

At 40% utilization: $13.9 / 0.40 \approx 34.7$ days

Lesson

Peak FLOP/s is not sustained application performance.

Worked example: fixed compute budget

The same compute can train different model and token combinations.

$$\text{Budget } C = 6 \times 10^{20} \text{ FLOPs}$$

Plan A: larger model

$$\begin{aligned} N &= 10\text{B} \\ D &= C/(6N) = 10\text{B tokens} \end{aligned}$$

Plan B: more data

$$\begin{aligned} N &= 2\text{B} \\ D &= C/(6N) = 50\text{B tokens} \end{aligned}$$

THINK Which plan should generalize better? What evidence is missing?

Compute-optimal scaling intuition

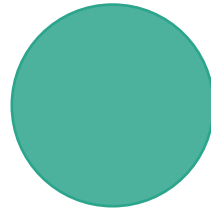
Bigger is not automatically better under a fixed budget.

Too small



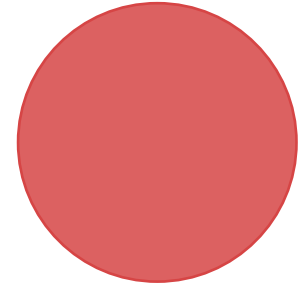
many tokens, limited capacity

Balanced



model and data grow together

Too large



too few tokens, undertrained

THINK Scaling law results guide resource allocation; they do not replace evaluation.

03

Hardware bottlenecks

FLOPs, capacity, bandwidth, and overhead.

Three fundamental limits

A workload may hit more than one.

Compute limit

Arithmetic units are busy.

Improve tensor-core use, precision, vectorization, or algorithmic FLOPs.

Memory-capacity limit

The tensors do not fit.

Reduce precision, batch, saved activations, or model storage.

Memory-bandwidth limit

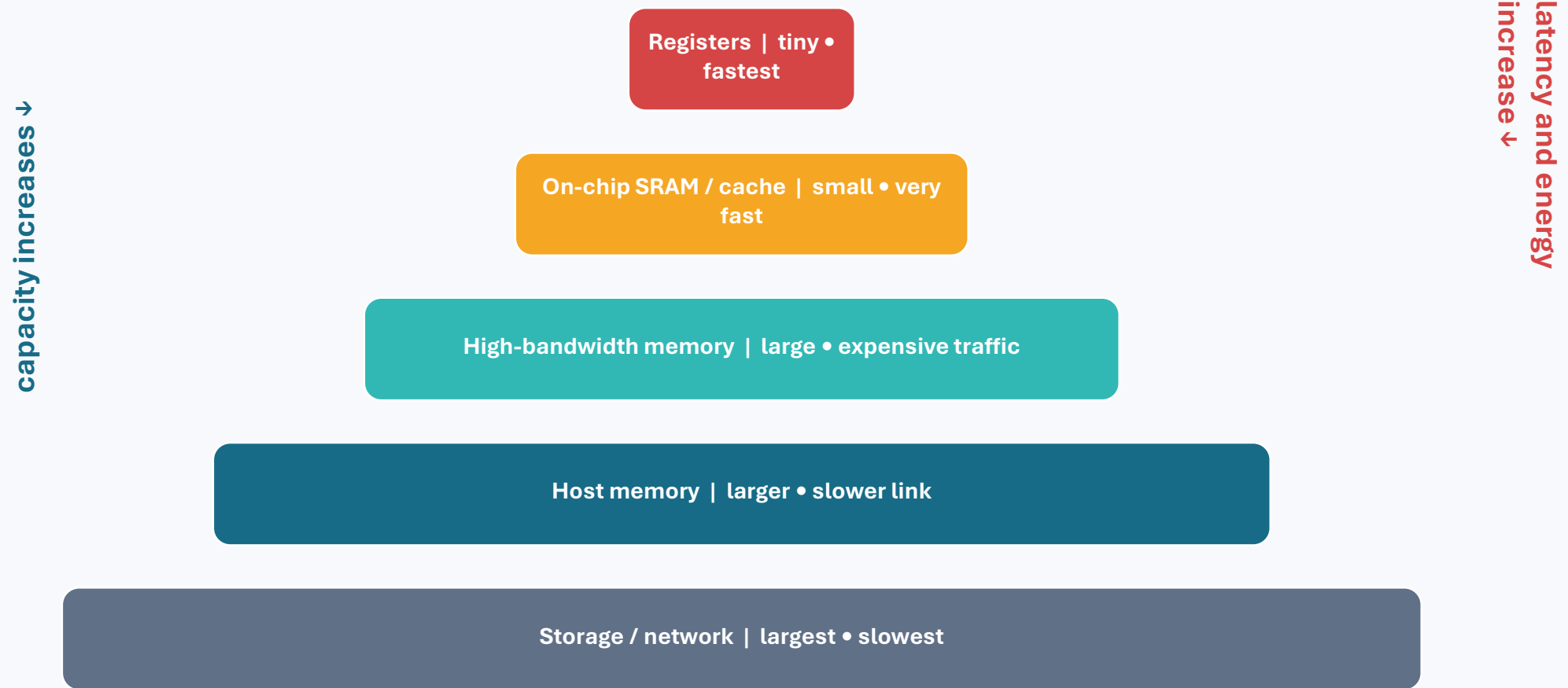
Compute waits for bytes.

Increase data reuse, tiling, fusion, or arithmetic intensity.

THINK Optimizing the wrong resource may produce no speedup.

Memory hierarchy

Closer storage is faster, smaller, and more reusable.



THINK The fastest byte is the byte you do not move.

Arithmetic intensity

How much useful math occurs per byte moved?

$$I = \text{FLOPs} / \text{bytes moved}$$

Low intensity

few operations per byte
likely bandwidth-bound

Ridge region

compute and bandwidth comparable

High intensity

substantial reuse
possibly compute-bound

THINK Tiling raises intensity by reusing data in fast memory.

Worked example: vector addition

$z = x + y$ is dominated by memory traffic.

Bytes per FP32 element

read x: 4
read y: 4
write z: 4

Arithmetic

1 addition
= 1 FLOP

Intensity

1 / 12
 ≈ 0.083 FLOP/byte

At 1 TB/s: bandwidth roof ≈ 0.083 TFLOP/s

Diagnosis

Adding arithmetic units will not fix the main bottleneck.

Worked example: matrix multiplication

Large tiled GEMM achieves high arithmetic intensity.

FLOPs

$C = AB$, $n \times n$ matrices
 $\approx 2n^3$

Minimum matrix traffic

$3n^2$ FP32 elements
 $\approx 12n^2$ bytes

Intensity

$2n^3 / 12n^2$
 $= n/6$

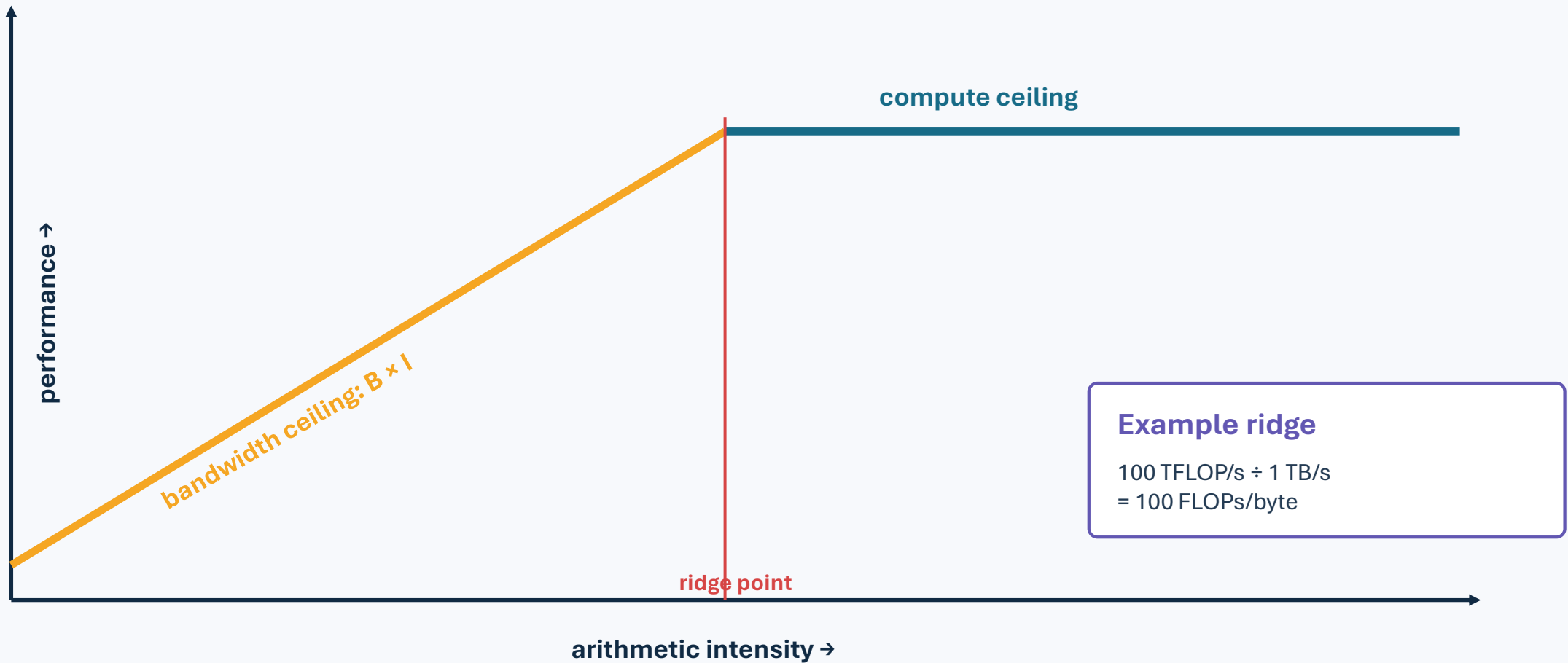
For $n = 1,024$: $I \approx 170.7$ FLOPs/byte

Why GPUs excel

Large matrix tiles repeatedly reuse data near the compute units.

Roofline reasoning

Achievable performance is limited by compute or bandwidth.



04

Profiling and optimization

Measure first, then target the limiting resource.

Optimization map

Choose an action that targets the diagnosed bottleneck.

Reduce FLOPs

smaller model, shorter context, sparsity

Increase compute efficiency

batching, vectorization, tensor-friendly shapes

Reduce traffic

tiling, fusion, reuse, data layout

Reduce capacity

lower precision, smaller microbatch, checkpointing

Improve input

prefetch, parallel loading, cached preprocessing

Reduce overhead

fewer kernel launches, larger operations

THINK An optimization may move the workload to a different bottleneck.

Scientific benchmarking protocol

A fast number is meaningless without a reproducible method.

1

Warm up before timing

2

Run multiple trials

3

Report median and variability

4

Synchronize asynchronous devices

5

Separate setup from execution

6

Keep shapes and precision fixed

7

Verify numerical correctness

8

Record hardware and software

9

Measure end-to-end and kernels

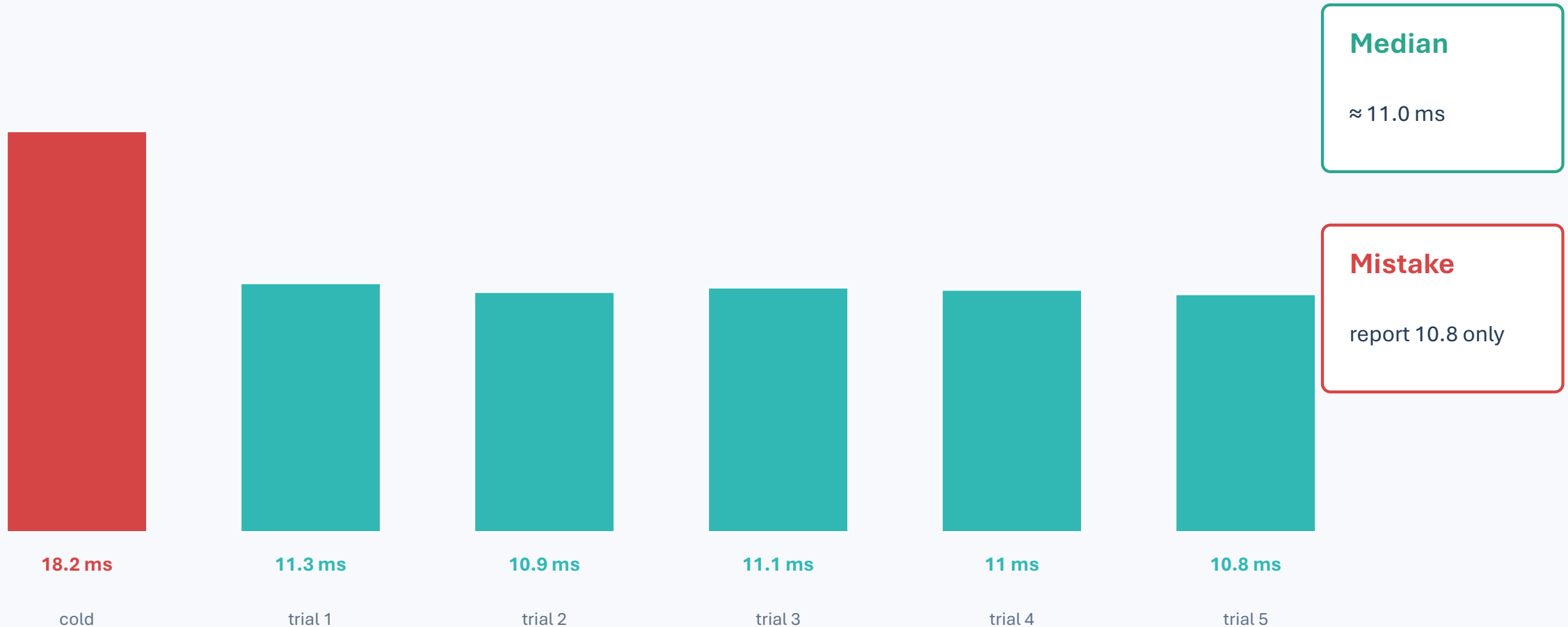
10

Compare equal quality targets

THINK Benchmark time-to-quality, not throughput alone.

Worked benchmarking example

Why one untimed warm-up and one measured run can mislead.



THINK Warm-up, caching, and compilation can change the first measurement.

Diagram: tensor lifecycle during training

The same tensor may move, transform, persist, or disappear.



Persistent

weights and optimizer states

Temporary

intermediates and workspaces

Saved

activations needed by backward

THINK Memory optimization starts by identifying each tensor's lifetime.

Diagram: where transformer compute goes

Projection and MLP matrix multiplications usually dominate dense-model FLOPs.



Short context

d^2 projection terms often dominate.

Long context

S^2 attention terms become increasingly important.

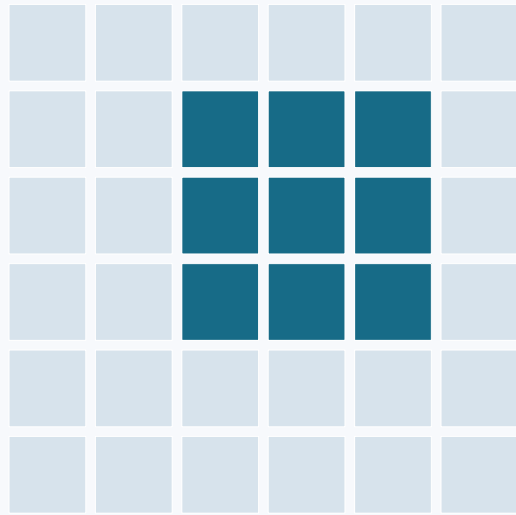
Planning rule

Count both model-width and context-length terms.

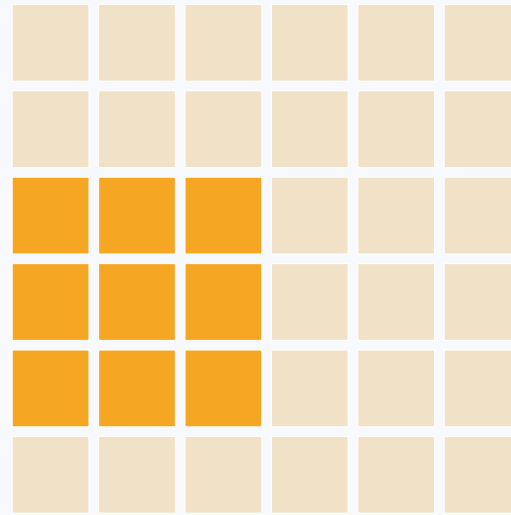
THINK At what sequence length does attention stop being “small”?

Diagram: tiling creates data reuse

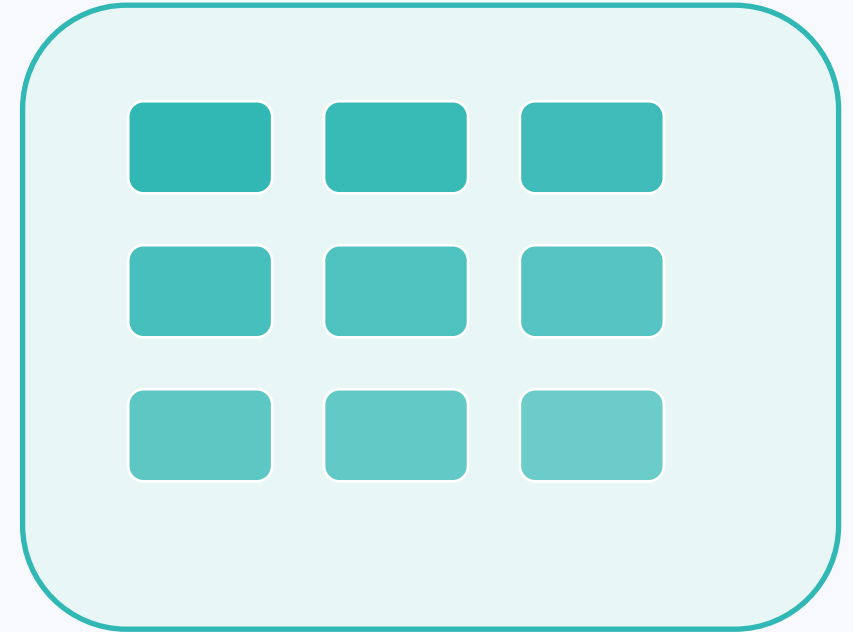
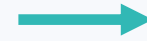
Load a small block into fast memory, then perform many operations on it.



A tile



B tile



**many multiply-adds
per loaded tile**

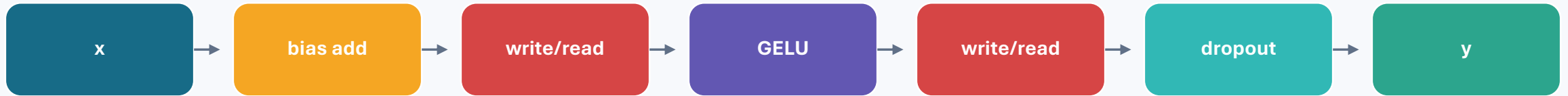
Performance effect

More reuse → fewer HBM transactions → higher arithmetic intensity

Diagram: kernel fusion reduces round trips

Combine adjacent elementwise operations when intermediate tensors need not return to HBM.

UNFUSED



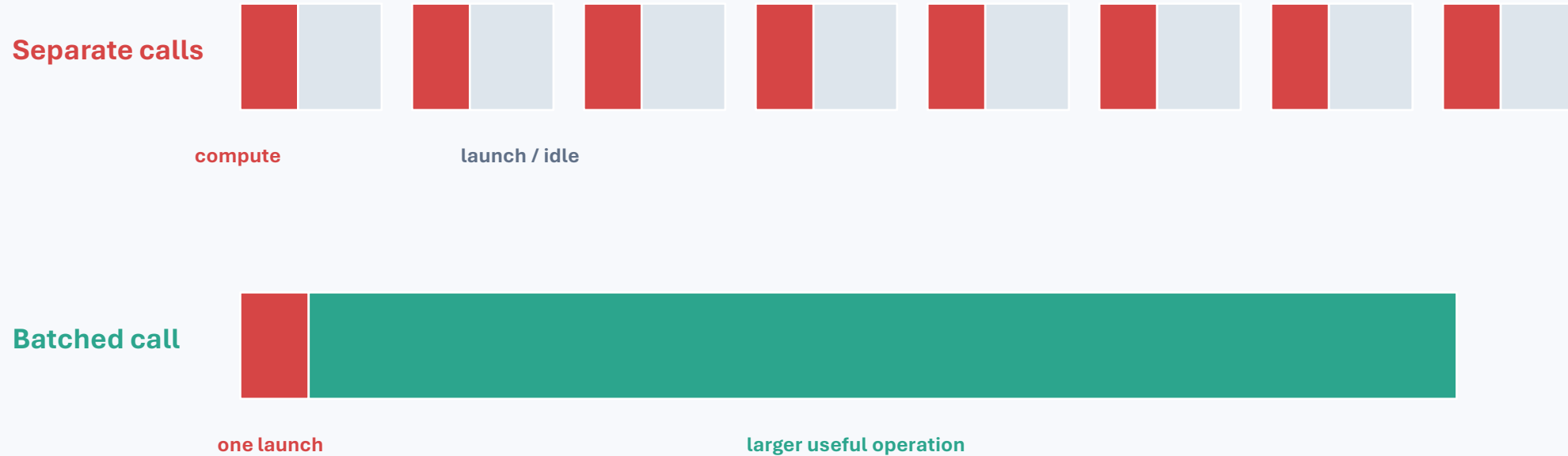
FUSED



THINK Fusion reduces memory traffic and launch overhead, but may increase implementation complexity.

Worked example: batching amortizes launch overhead

Many tiny operations can underuse hardware even when total FLOPs are unchanged.

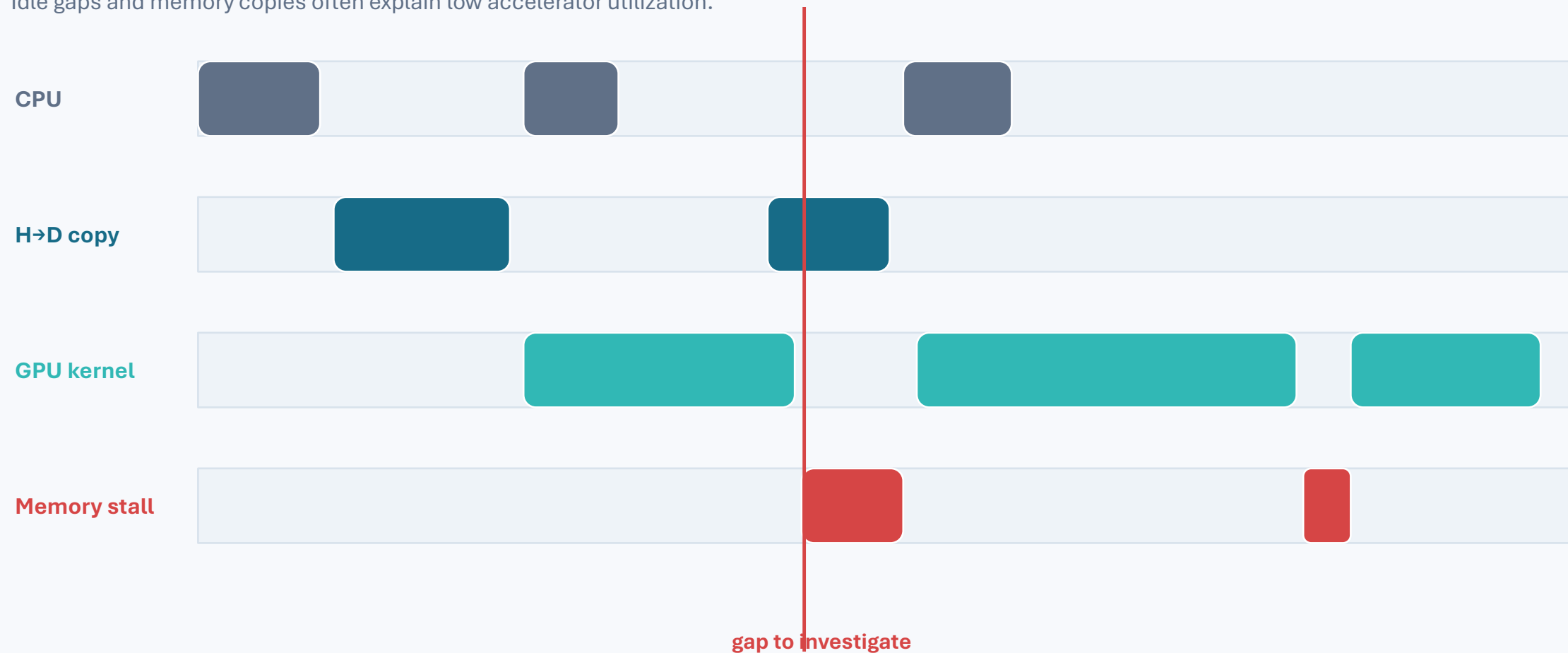


Tradeoff

Batching improves utilization, but raises latency and memory demand.

Diagram: read a profiler timeline

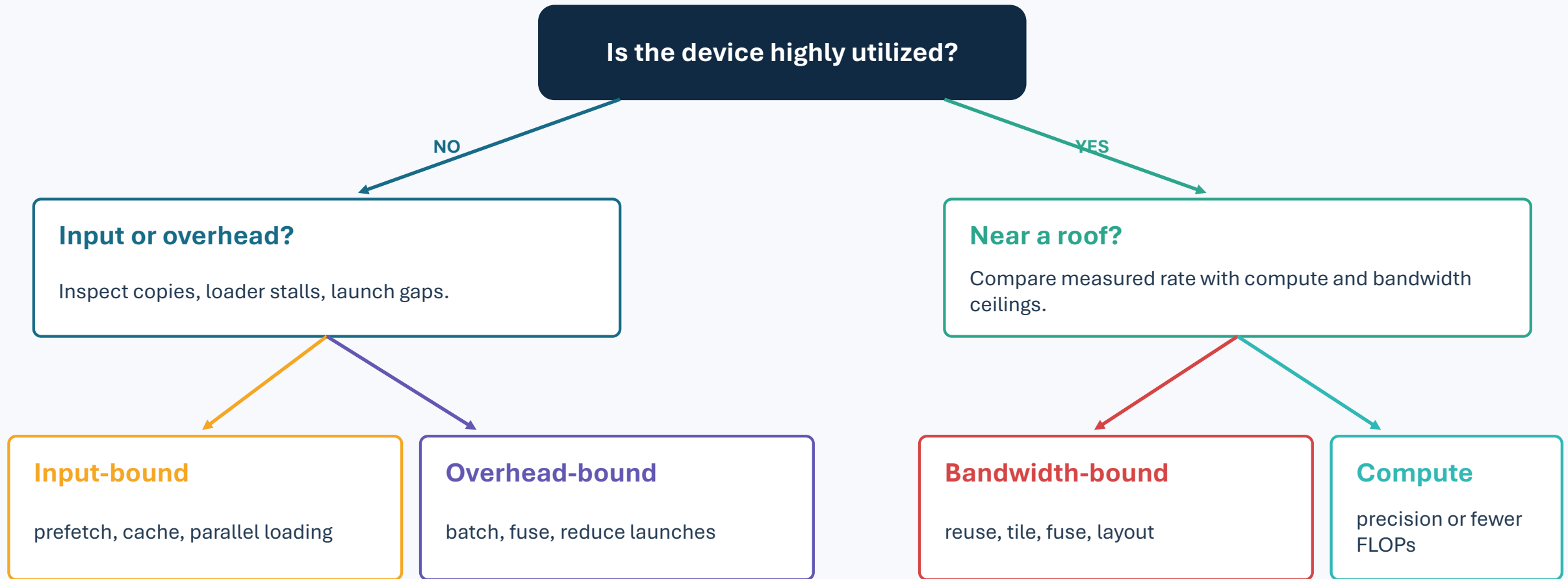
Idle gaps and memory copies often explain low accelerator utilization.



THINK The first profiling question is: what is the device waiting for?

Diagram: bottleneck decision tree

Use measurements to choose the next experiment.



THINK Repeat after each optimization because the bottleneck can move.

Worked solution scaffold: 3B model challenge

Separate exact arithmetic from engineering judgment.

1

Compute

$$6 \times 3B \times 60B = 1.08 \times 10^{21} \text{ FLOPs}$$

2

Ideal time

$$\div 150 \text{ TFLOP/s} = 7.2 \times 10^6 \text{ s} \approx 83.3 \text{ days}$$

3

BF16 weights

$$3B \times 2 \text{ bytes} \approx 6 \text{ GB}$$

4

Attention growth

doubling context makes score terms 4× larger

5

Judgment

training time, activations, and bandwidth still matter

THINK The model weights fit, but training memory and elapsed time may not.

Five ideas to remember

- 1 Count parameters and bytes before discussing speed.
- 2 Use $C \approx 6ND$ for high-level training-compute planning.
- 3 FLOPs alone do not capture memory traffic or overhead.
- 4 Arithmetic intensity separates bandwidth and compute regimes.
- 5 **Profile scientifically, then optimize the limiting resource.**